Document 1039

Section 1

Software engineering is a branch of both computer science and engineering focused on designing, developing, testing, and maintaining software applications. It involves applying engineering principles and computer programming expertise to develop software systems that meet user needs.
The terms programmer and coder overlap software engineer, but they imply only the construction aspect of a typical software engineer workload.
A software engineer applies a software development process, which involves defining, implementing, testing, managing, and maintaining software systems, as well as developing the software development process itself.

Beginning in the 1960s, software engineering was recognized as a separate field of engineering.
The development of software engineering was seen as a struggle. Problems included software that was over budget, exceeded deadlines, required extensive debugging and maintenance, and unsuccessfully met the needs of consumers or was never even completed.
In 1968, NATO held the first software engineering conference, where issues related to software were addressed. Guidelines and best practices for the development of software were established.
The origins of the term software engineering have been attributed to various sources. The term appeared in a list of services offered by companies in the June 1965 issue of "Computers and Automation" and was used more formally in the August 1966 issue of Communications of the ACM (Volume 9, number 8) in "President's Letter to the ACM Membership" by Anthony A. Oettinger. It is also associated with the title of a NATO conference in 1968 by Professor Friedrich L. Bauer. Margaret Hamilton described the discipline of "software engineering" during the Apollo missions to give what they were doing legitimacy. At the time, there was perceived to be a "software crisis". The 40th International Conference on Software Engineering (ICSE 2018) celebrates 50 years of "Software Engineering" with the Plenary Sessions' keynotes of Frederick Brooks and Margaret Hamilton.
In 1984, the Software Engineering Institute (SEI) was established as a federally funded research and development center headquartered on the campus of Carnegie Mellon University in Pittsburgh, Pennsylvania, United States.
Watts Humphrey founded the SEI Software Process Program, aimed at understanding and managing the software engineering process. The Process Maturity Levels introduced became the Capability Maturity Model Integration for Development (CMMI-DEV), which defined how the US Government evaluates the abilities of a software development team.
Modern, generally accepted best practices for software engineering have been collected by the ISO/IEC JTC 1/SC 7 subcommittee and published as the Software Engineering Body of

Knowledge (SWEBOK). Software engineering is considered one of the major computing disciplines.
In modern systems, where concepts such as Edge Computing, Internet of Things and Cyber-physical Systems are prevalent, software is a critical factor. Thus, software engineering is closely related to the Systems Engineering discipline. The Systems Engineering Body of Knowledge claims:

Software is prominent in most modern systems architectures and is often the primary means for integrating complex system components. Software engineering and systems engineering are not merely related disciplines; they are intimately intertwined....Good systems engineering is a key factor in enabling good software engineering.

Section 2

=== Definition ===
Notable definitions of software engineering include:

"The systematic application of scientific and technological knowledge, methods, and experience to the design, implementation, testing, and documentation of software."—The Bureau of Labor Statistics—IEEE Systems and software engineering – Vocabulary
"The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software."—IEEE Standard Glossary of Software Engineering Terminology
"An engineering discipline that is concerned with all aspects of software production."—Ian Sommerville
"The establishment and use of sound engineering principles in order to economically obtain software that is reliable and works efficiently on real machines."—Fritz Bauer
"A branch of computer science that deals with the design, implementation, and maintenance of complex computer programs."—Merriam-Webster
"'Software engineering' encompasses not just the act of writing code, but all of the tools and processes an organization uses to build and maintain that code over time. [...] Software engineering can be thought of as 'programming integrated over time.'"—Software Engineering at Google
The term has also been used less formally:

as the informal contemporary term for the broad range of activities that were formerly called computer programming and systems analysis
as the broad term for all aspects of the practice of computer programming, as opposed to the theory of computer programming, which is formally studied as a sub-discipline of computer science
as the term embodying the advocacy of a specific approach to computer programming, one that urges that it be treated as an engineering discipline rather than an art or a craft, and advocates the codification of recommended practices

Section 3

=== Suitability ===
Individual commentators have disagreed sharply on how to define software engineering or its legitimacy as an engineering discipline. David Parnas has said that software engineering is, in fact, a form of engineering. Steve McConnell has said that it is not, but that it should be. Donald Knuth has said that programming is an art and a science. Edsger W. Dijkstra claimed that the terms software engineering and software engineer have been misused in the United States.

Requirements engineering is about elicitation, analysis, specification, and validation of requirements for software. Software requirements can be functional, non-functional or domain.
Functional requirements describe expected behaviors (i.e. outputs). Non-functional requirements specify issues like portability, security, maintainability, reliability, scalability, performance, reusability, and flexibility. They are classified into the following types: interface constraints, performance constraints (such as response time, security, storage space, etc.), operating constraints, life cycle constraints (maintainability, portability, etc.), and economic constraints. Knowledge of how the system or software works is needed when it comes to specifying non-functional requirements. Domain requirements have to do with the characteristic of a certain category or domain of projects.

Software design is the process of making high-level plans for the software. Design is sometimes divided into levels:

Section 4

Interface design plans the interaction between a system and its environment as well as the inner workings of the system.
Architectural design plans the major components of a system, including their responsibilities, properties, and interfaces between them.
Detailed design plans internal elements, including their properties, relationships, algorithms and data structures.

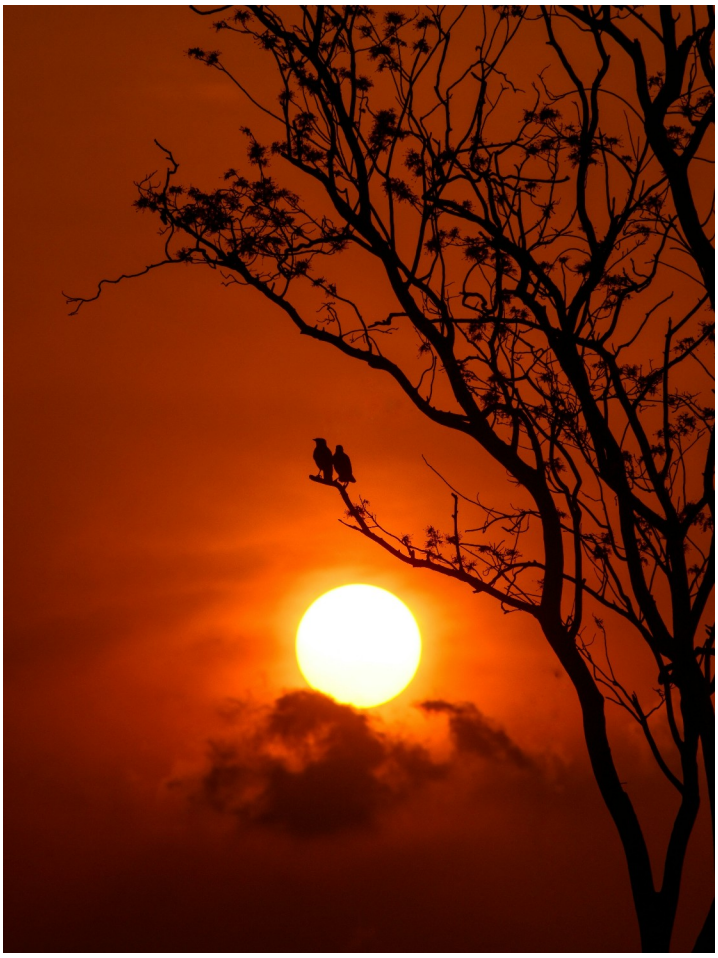
Software construction typically involves programming (a.k.a. coding), unit testing, integration testing, and debugging so as to implement the design."Software testing is related to, but different from, ... debugging".

Software testing is an empirical, technical investigation conducted to provide stakeholders with information about the quality of the software under test. Software testing can be viewed as a risk based activity.
When described separately from construction, testing typically is performed by test engineers or quality assurance instead of the programmers who wrote it. It is performed at the system level and is considered an aspect of software quality. The testers' goals during

the testing process are to minimize the overall number of tests to a manageable set and make well-informed decisions regarding which risks should be prioritized for testing and which can wait.

Section 5

Program analysis is the process of analyzing computer programs with respect to an aspect such as performance, robustness, and security.

Software maintenance refers to supporting the software after release. It may include but is not limited to: error correction, optimization, deletion of unused and discarded features, and enhancement of existing features.
Usually, maintenance takes up 40% to 80% of project cost.

Knowledge of computer programming is a prerequisite for becoming a software engineer. In 2004, the IEEE Computer Society produced the SWEBOK, which has been published as ISO/IEC Technical Report 1979:2005, describing the body of knowledge that they recommend to be mastered by a graduate software engineer with four years of experience. Many software engineers enter the profession by obtaining a university degree or training at a vocational school. One standard international curriculum for undergraduate software engineering degrees was defined by the Joint Task Force on Computing Curricula of the IEEE Computer Society and the Association for Computing Machinery, and updated in 2014. A number of universities have Software Engineering degree programs; as of 2010, there were 244 Campus Bachelor of Software Engineering programs, 70 Online programs, 230 Masters-level programs, 41 Doctorate-level programs, and 69 Certificate-level programs in the United States.
In addition to university education, many companies sponsor internships for students wishing to pursue careers in information technology. These internships can introduce the student to real-world tasks that typical software engineers encounter every day. Similar experience can be gained through military service in software engineering.